# From Static to Contextual

## Distributional Semantics and What's Inside BERT

From distributional co-occurrence to contextual syntax

Yellapu Dheeraj Babu
23CS10081

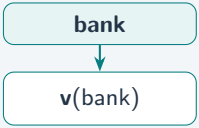
NLP CS60075 | Assignment II – Project 5
Indian Institute of Technology Kharagpur

10-minute project presentation

November 2026

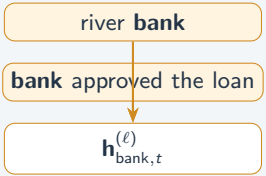
# Why compare static and contextual representations?

## Static word vectors




- One vector per word type.
- Meaning is inferred from aggregate co-occurrence.
- Efficient, but cannot distinguish a word's different uses.

## Contextual BERT representations




- A different vector for each token occurrence.
- Encodes left and right sentence context.
- Can carry syntax and sense information in its layers.

## The central question

Does information useful for word meaning also preserve token-to-token syntax?

## Problem statement and experimental questions

### Problem statement

Study the transition from classical static vectors to contextual BERT representations, and test what happens to semantic and syntactic information when contextual vectors are pooled into one vector per word.

### Task 1 – Baseline

**Build PPMI + SVD vectors.**

- Learn from EWT co-occurrence.
- Evaluate semantic similarity and analogies.
- Check cosine-MST edges against gold trees.

### Task 2 – Reproduction

**Probe frozen BERT.**

- Learn a linear geometry for tree distance and depth.
- Sweep layers and probe ranks.
- Select the strongest syntax layer.

### Task 3 – Combine and improve

**Pool BERT into static vectors.**

- Average the selected layer by word type.
- Re-run the same semantic tests.
- Compare tree recovery with Task 1.

### What is being tested?

A representation may be good at ranking semantically similar word types without being good at identifying which word tokens are syntactically connected.

# Dataset and evaluation protocol

## UD English EWT

- Web-genre English: reviews, emails, newsgroups, Q&A and blogs.
- Gold Universal Dependencies trees provide token heads for syntax evaluation.
- Raw tokens train static vectors; gold heads supply syntax supervision/evaluation.

| Split | Sentences | Words |
|---|---|---|
| Train | 12,544 | 204,578 |
| Dev | 2,001 | 25,148 |
| Test | 2,077 | 25,094 |

Dataset splits and fields: UD English EWT, assignment dataset specification and experiment report (November 2026).

## Evaluation sets and metrics

- **WordSim-353:** Spearman $\rho$ between model cosine scores and human similarity rankings.
- **Google analogies:** 3CosAdd accuracy; exclude the three query words.
- **Dependency trees:** UUAS after decoding a minimum spanning tree (MST).
- **Structural probe:** depth Spearman alongside UUAS.

**Coverage:** 248/353 WordSim pairs (70.3%) and 5,727/19,544 analogy questions (29.3%) are fully in-vocabulary.

# Code organisation and how to run

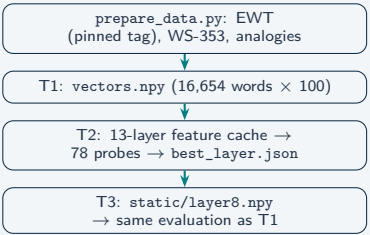
## One folder per task

| | |
|---|---|
| `code/common/` | data, MST, bootstrap, **shared evaluation** |
| `code/task1/` | co-occurrence → PPMI → SVD |
| `code/task2/` | BERT features → probes → $\ell^*$ |
| `code/task3/` | pool layer $\ell^*$ → same evaluation |
| `code/run_all.py` | every step, in order, one log each |
| `run_all_colab.ipynb` | end-to-end run on Colab |

## Design choice

T1 and T3 call *the same* evaluation function on the same vocabulary, so their comparison uses an identical protocol by construction.

## Data flow




`python code/run_all.py` · seed 42 everywhere

# Task 1 methodology: PPMI + truncated SVD

1. Count word–context co-occurrences inside each training sentence, using a symmetric window of five words on either side.
2. Convert raw counts to positive pointwise mutual information:

$$\mathrm{PPMI}(w, c) = \max\left(0, \log \frac{P(w, c)}{P(w)P(c)}\right).$$

3. Apply truncated SVD and retain 100 dimensions; use $U\Sigma$ as the word vectors.
4. Compute cosine similarity for WordSim-353; answer analogies with 3CosAdd:

$$\arg\max_{x} \cos(\mathbf{x}, \mathbf{b} - \mathbf{a} + \mathbf{c}).$$

## Structural bonus check

For each sampled test sentence:

- Compute pairwise cosine distances between word vectors.
- Decode an MST using Prim's algorithm.
- Compare undirected edges with the gold dependency tree.

**Metric:** UUAS = correct undirected edges / gold edges.

## Why this is a useful baseline

PPMI weights above-chance co-occurrence; SVD compresses it. Sparse counts remain the main limitation.

# Task 1 results: the small training corpus is a bottleneck

## Required PPMI-SVD configuration

| Metric | Covered items | All items |
|---|---|---|
| WordSim-353 Spearman $\rho$ | 0.104 | 0.048 |
| Google analogy accuracy | 0.51% | 0.15% |
| MST UUAS (20 sentences) | 20.5% | – |

- WordSim correlation rises to 0.317 for the 32 covered pairs whose two words occur at least 20 times.
- The best tested PPMI variant reaches only $\rho = 0.134$; hyperparameter changes do not solve analogy sparsity.
- Frequent-word neighbours are often co-occurring partners, but rare-word neighbours are noisy.

## Tree-recovery baselines

**20.5%**
PPMI-SVD, 20-sentence sample

**15.2%**
expected random tree

**43.2%**
linear chain baseline

## Interpretation

The baseline beats random but trails word order; sparse EWT counts limit lexical estimates and four-word analogies.

# Task 2 methodology: a structural probe on frozen BERT

## Representations and probe

- Feed each sentence to frozen `bert-base-uncased`; retain layer 0 (embeddings) and transformer layers 1–12.
- Each token has a 768-dimensional representation $\mathbf{h}_i^{(\ell)}$.
- Learn $B_\ell \in \mathbb{R}^{k \times 768}$ per layer, with $k \in \{32, 64, 128\}$.

## Two geometric objectives

**Tree distance:**

$$d_B(i,j) = \|B_\ell h_i - B_\ell h_j\|_2^2$$

**Tree depth:**

$$\text{depth}_B(i) = \|B_\ell h_i\|_2^2$$

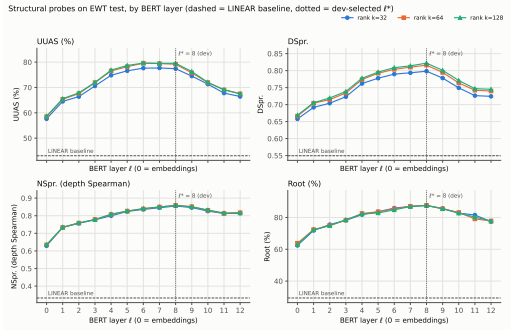
## Training and decoding

- Gold distances and depths are computed from EWT `head`.
- Train distance and depth probes for every layer/rank.
- Adam (initial LR $10^{-3}$), batch 20, 30 epochs; LR $\times 0.1$ when dev loss stalls (paper); keep the best dev checkpoint.
- Decode an MST from predicted distances.

## Reported measures

**UUAS** evaluates recovered dependency edges; **depth Spearman** evaluates whether predicted norms rank tokens by gold tree depth.

# Task 2 results: syntax peaks in the middle layers




UUAS, depth Spearman and root prediction by BERT layer; dotted line marks the dev-selected layer.

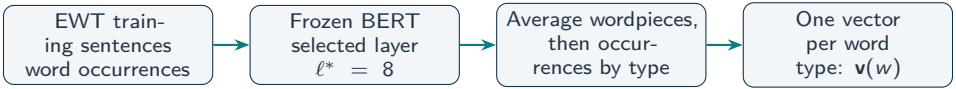
**79.5%**
test UUAS at layer 8 (paper: 79.8%)

**0.859**
depth Spearman at layer 8 (paper: 0.87)

**58.7% → 79.5%**
layer 0 to layer 8 UUAS

- Layers 6–8 tie statistically; gains saturate by $k = 64$.
- Layer 12 falls to 67.6% UUAS.
- Paper: PTB/WSJ + cased = 79.8%; EWT + uncased = 79.5%.

# Task 3 methodology: turn contextual BERT states into static vectors




## Pooling operation

For all occurrences $t$ of word type $w$ in EWT train:

$$\mathbf{v}(w) = \frac{1}{N_w} \sum_{t:w_t=w} \mathbf{h}_t^{(8)}.$$

The same lowercased vocabulary and benchmark protocol as Task 1 are used.

## Post-processing and fairness checks

- Also test all-but-the-top (ABTT): subtract the mean and remove the top 7 principal directions from BERT vectors.
- Apply one common evaluation function to T1 and T3.
- Reuse the exact same 20 sampled sentences and expanded 660-sentence pool.

## Important comparison caveat

T3 inherits BERT pretraining on roughly 3.3B words; T1 sees only 204.6K EWT words. This is not a same-data comparison.

# Task 3 results: pooled BERT vectors transform semantic performance

## Comparison: semantic metrics (covered/all) and tree recovery (pool)

| Representation | WordSim $\rho$ cov. / all | Analogy (%) cov. / all | MST UUAS pool |
|---|---|---|---|
| T1: PPMI-SVD (required) | 0.104 / 0.048 | 0.51 / 0.15 | 24.8% |
| T3: BERT-static, layer 8 | 0.577 / 0.452 | 67.7 / 19.8 | 17.9% |
| T3 + ABTT | **0.682 / 0.539** | **71.0 / 20.8** | 19.6% |

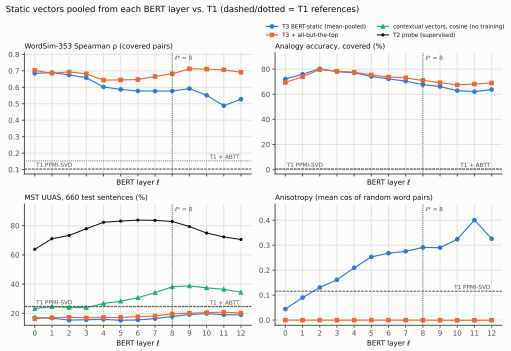
## Large semantic gains

- WordSim gain: $+0.474$ for raw T3 and $+0.579$ with ABTT.
- ABTT's WordSim gain over T1 has 95% CI $[0.46, 0.71]$.
- On analogies, T3 solves 3,848 cases T1 misses; none are solved only by T1 (McNemar $p < 10^{-300}$).

## Qualitative behaviour

- For *good*, T3 returns *excellent, great, nice, decent, wonderful*.
- Strong on plural morphology (96.6%) and capitals (93.6%); weak on currency (0/28) and opposites (26.9%).

# The best layer depends on what we want to preserve




## Different optima

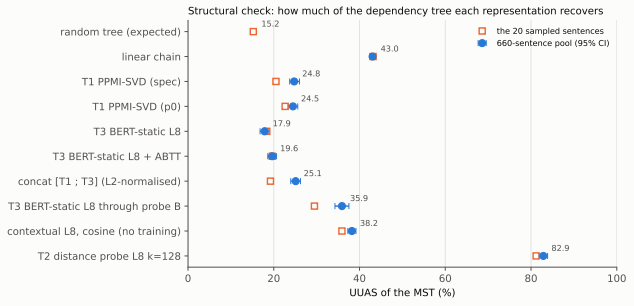
- WordSim-353 peaks at layer 1: $\rho = 0.690$.
- Analogy accuracy peaks at layer 2: 80.3%.
- Syntax probe peaks around layers 6–8; layer 8 is dev-selected.

## Why layer 8?

It is selected for its syntactic structure, not because it is the best layer for static lexical semantics.

Layer-wise analysis is from the experiment report; contextual-to-static layer trends are consistent with Bommasani et al. (ACL 2020).

# Task 3 results: semantic gains do not transfer to tree recovery




## 660-sentence pool

| Representation | UUAS |
|---|---|
| Random tree | 17.1% |
| T3 static, layer 8 | 17.9% |
| T3 + ABTT | 19.6% |
| T1 PPMI-SVD | 24.8% |
| Contextual cosine | 38.2% |
| Linear chain | 43.0% |
| T2 structural probe | **82.9%** |

**Takeaway:** The 20-sentence T1–T3 difference is not significant; on the 660-sentence pool T3 is 6.9 points below T1 (95% CI $[-8.0, -5.8]$). The probe reaches 82.9% by learning syntax-specific token geometry.

# Why the evaluations disagree: substitutes are not syntactic partners

## Paradigmatic similarity: T3

- 74.0% of T3 top-10 neighbours share its majority POS (T1: 32.6%).
- Example: *car* → *cars*, *vehicle*, *vehicles*, *SUV*, *Ferrari*.
- These substitutes help lexical similarity.

## Syntagmatic association: T1

- T1 neighbours co-occur within ±5 tokens 55.7% of the time (T3: 16.2%).
- Example: *car* → *home*, *guy*, *test*, *driving*, *drive*.
- Co-occurrence can reflect sentence-level partners.

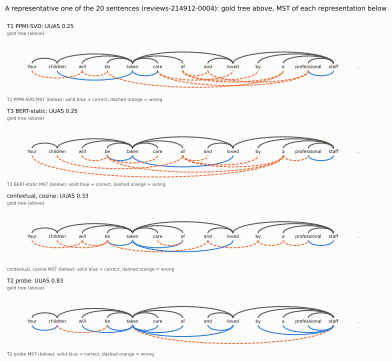
## Direct structural diagnostics

| Diagnostic | T1 | T3 |
|---|---|---|
| Gold edge closer than non-edge (AUC) | 0.54 | 0.46 |
| Function-word mean MST degree | 1.84 | 2.29 |
| Same-UPOS nearest neighbours | 0.33 | 0.74 |

## Mechanism

Averaging retains dictionary-like meaning but removes which head or dependent a particular token attaches to.

# A sentence-level example makes the loss visible




## Example sentence

*"Your children will be taken care of and loved by a professional staff."*

- T1 PPMI-SVD: 25% UUAS.
- T3 BERT-static layer 8: 25% UUAS.
- Raw contextual cosine: 33% UUAS.
- T2 trained structural probe: **83% UUAS**.

## What to notice

Pooled vectors produce long, incorrect links; the learned probe recovers more gold edges through a syntax-aligned projection.

Representative sentence selected as the example whose per-method scores were closest to each method's average among the 20 sampled sentences.

# Reproducibility, protocol deviations and limitations

## Reproducibility controls

- Seed 42 for sampling, SVD, bootstraps and experiments.
- UD 2.18 loader tag; BERT checkpoint `bert-base-uncased`, revision `86b5e09`.
- Python 3.12, PyTorch 2.15, Transformers 5.22, Datasets 5.1; 2 NVIDIA L40S GPUs.
- Runtime: BERT features 7 min; 78 probes about 49 + 54 min on two GPUs.

## Important deviations and checks

- Probe used learning-rate decay and best-dev selection, following the paper.
- Constant $10^{-3}$ LR / epoch-30 control retains layer 8 but reaches only 62.0% UUAS (non-convergence).
- Layer 0 is the non-contextual baseline; 660 sentences supplement the required 20-sentence sample.

## Limitations

One treebank and one BERT model; unequal pretraining resources; a selected in-vocabulary sentence pool; and an under-powered 20-sentence check. Causes of the small paper gap are plausible, not isolated.

## Conclusions: what did the experiments establish?

1. **Classical distributional vectors are data-hungry.** PPMI-SVD trained on 204.6K EWT words provides only weak semantic benchmarks and limited tree recovery.
2. **BERT contains recoverable syntax.** A linear structural probe reaches 79.5% UUAS on EWT test, with the strongest geometry around layers 6–8.
3. **Pooling helps lexical semantics, not syntax.** Layer-8 BERT-static vectors raise WordSim-353 correlation from 0.104 to 0.577 (0.682 with ABTT), but reduce pool UUAS from 24.8% to 17.9%.
4. **The objectives are different.** Semantic benchmarks reward paradigmatic similarity between word types; dependency recovery needs token-specific, syntagmatic relations in context.

**A good static meaning vector is not automatically a good syntactic representation.**

**Thank you** Questions?

# Selected references


- Z. S. Harris (1954). *Distributional structure*. *Word*, 10(2–3).
- J. Devlin, M.-W. Chang, K. Lee, K. Toutanova (2019). *BERT: Pre-training of deep bidirectional transformers for language understanding*. NAACL-HLT. doi:10.18653/v1/N19-1423.
- O. Levy and Y. Goldberg (2014). *Neural word embedding as implicit matrix factorization*. NeurIPS. paper.
- J. Hewitt and C. D. Manning (2019). *A structural probe for finding syntax in word representations*. NAACL-HLT, 4129–4138. doi:10.18653/v1/N19-1419.
- R. Bommasani, K. Davis, C. Cardie (2020). *Interpreting pretrained contextualized representations via reductions to static embeddings*. ACL, 4758–4781. doi:10.18653/v1/2020.acl-main.431.
- J. Mu and P. Viswanath (2018). *All-but-the-top: Simple and effective postprocessing for word representations*. ICLR.
- M. Sahlgren (2006). *The Word-Space Model*. Ph.D. thesis, Stockholm University.
- M.-C. de Marneffe, C. D. Manning, J. Nivre, D. Zeman (2021). *Universal Dependencies*. *Computational Linguistics*, 47(2).


Numerical results, experiment settings, plots and qualitative examples are from the supplied Project 5 report and its generated tables/figures. Assignment framing and dataset requirements follow the supplied CS60075 Project 5 and UD English EWT dataset specifications.